

Thatchaphon Panyajamorn 6781578

A02 Financial Tracker Report

ICCS 121 System Skill and Low-Level Programming

Professor: Songpon Teerakanok

19.06.2026

Project Overview

This C program implements a **linked list** data structure in a real-world application, which is a **financial tracker** where you can track your income and expenses. It allows you to manage your money more efficiently by monitoring your income and spending. You can add income and expenses with a description. Moreover, you can print all the transactions and will be able to save them in a transactions log file once you quit the program. Besides, this program is flexible enough to delete transactions or insert at any position you want.

Objectives

- **C Programming Fundamentals:** Write a C program that can be used in a real-world application.
- **Linked List Data Structure:** Apply the singly linked list data structure to manage dynamic transaction records.
- **File Handling:** Become familiar with file handling, particularly in file pointers and fopen to read and write data.
- **Makefile:** Implement a script for compiling C files in a faster way.

System Design

Data Structure

This program heavily relies on a certain data type: Node, which is a user-defined data type. It contains six important data types, such as:

- float data (to contain a transaction)
- int is_expense (to specify if it is an income or expense (1 = expense, 0 = income))
- int is_saved (to specify if it has been saved for different output before and after printing the transactions) 1 = saved, 0 = unsaved
- int is_delete (to specify its status of existence) 1 = deleted, 0 = not deleted
- char description[100]: This variable contains a description of a transaction.
- Node* next: it contains the next node.

A linked list is more practical in this program over an array because we can manage the data dynamically.

```
typedef struct Node {
    float data;
    int is_expense; // 1 = expense, 0 = income
    int is_saved; // 1 = saved, 0 for new
    int is_deleted; // 1 = deleted 0 = not deleted
    char description[100];
    struct Node* next;
} Node;
```

Core Function

There are nine functions in total for this program:

- **Node* createNode(float data)**: This function does memory allocation for creating a new node with an initial value initialized. It returns a new node.

```
Node* createNode(float data){
    Node* newNode=(Node*)malloc(sizeof(Node));
    if (newNode == NULL){
        printf("Allocation failed\n");
        exit(0);
    }
    newNode -> data = data;
    newNode -> next = NULL;
    newNode -> is_saved = 0;
    newNode -> is_deleted = 0;

    return newNode;
}
```

- **char * BudgetStatus(float *balance, float *expense)**: This function monitors the balance. if balance is greater than zero, then it returns "within budget". If not, that means it is below zero (expense is more than income), so it returns "Over Budget!".

```
char* budgetStatus(float *balance, float *expense){
    if(*balance >= 0){
        return "Within Budget";
    } else{
        return "Over Budget!";
    }
}
```

- **Void insert_at_position(Node** head_ref, Node* new_node, int position)**: a helper function that is used when we want to insert an income or an expense at any desired position. It basically changes the next node of the head and a new node (new transaction) to any specified node for inserting at a certain point (according to a singly linked list structure where a certain node points to a new node and the other one gets a new link to the head node, so it changed position) using conditions and traversing.

```
void insert_at_position(Node** head_ref, Node* new_node, int position){
    // Case 1: Inserting at the very front (Position 1)
    if (position == 1 || *head_ref == NULL){
        new_node -> next = *head_ref;
        *head_ref = new_node;
        return;
    }
    // Case 2: Inserting in the middle or at the end
    Node* temp = *head_ref;
    for(int i = 1; temp->next != NULL && i < position - 1; i++){
        temp = temp -> next;
    }
    // Link the new node into the list
    new_node -> next = temp -> next;
    temp->next = new_node;
}
```

- **void add_income(Node** head_ref, int position, float income, float* balance, char* description)**: This function adds a new node of income. You can also add a description of this transaction. It will use a helper function <insert_at_position()> to select which position to be inserted, as we have created a node for this transaction. Lastly, it will add that income to the balance.

```
void add_income(Node** head_ref, int position, float income, float* balance, char* description){
    Node* new_node = createNode(income);
    new_node->next = NULL;
    new_node->is_expense = 0;
    strcpy(new_node->description, description);

    // Call insertion helper
    insert_at_position(head_ref, new_node, position);
    *balance += income;
}
```

- **void add_expense(Node** head_ref, int position, float expense, float *balance, char* description)**: This function does the same as add_income, it only change from income to an expense. It also subtracts the amount of the balance at the very end of this function.

```
void add_expense(Node** head_ref, int position, float expense, float *balance, char* description){
    Node* new_node = createNode(-expense);
    new_node->next = NULL;
    strcpy(new_node->description, description);
    new_node->is_expense = 1;

    // Call insertion helper
    insert_at_position(head_ref, new_node, position);

    *balance -= expense;
}
```

- **void delete(Node** head_ref, int position, float *balance):** This function deletes a certain node or transaction by traversing to the certain node and changing its previous node's link to the next node. After the deletion, one property, which is `is_deleted`, will be assigned to 1 to indicate that this transaction has been deleted and will be used in the `print()` function.

```
void delete(Node** head_ref, int position, float *balance){
    // check if the list is empty
    if (*head_ref == NULL || position < 1){
        printf("Error: Invalid position or empty list!\n");
        return;
    }

    Node* temp = *head_ref;

    for (int i = 1; temp != NULL && i < position; i++){
        temp = temp -> next;
    }

    if (temp == NULL){
        printf("Error: Position out of bounds!\n");
        return;
    }

    if(temp -> is_deleted){
        printf("Error: Transaction already deleted.\n");
        return;
    }

    // Mark for deletion
    temp -> is_deleted = 1;

    // Update balance
    if(temp -> is_expense){
        *balance += (-temp -> data);
    } else{
        *balance -= temp->data;
    }
}
```

- **void print(Node* node):** This function goes through all the nodes (transactions) and prints all of them to the screen. There are conditions toward print with special status, such as `---` d, `+++` i, and saved for deleted transactions, new transactions, and saved transactions, respectively.

```
void print(Node* node){
    int i = 1;
    printf("[ Transactions ]\n");
    while (node != NULL){
        char* status;
        if(node->is_deleted){
            status = "--- d";
        } else if(!node -> is_saved){
            status = "+++ i";
        } else{
            status = "(saved)";
        }
        printf("%d. %-20s%+8.2f%15s\n", i, node->description, node->data, status);
        i++;

        node = node -> next;
    }
}
```

- `void run_session(Node** head_node, float* balance, float* expense, FILE** fptr)`: This is a function that makes our program cleaner, as there will be 3-4 repetitions in our program for running the fintrack; this function replaced all of them.

```
void run_session(Node** head_node, float* balance, float* expense, FILE** fptr){
    int running = 1; // flag
    char description[100];
    float amount = 0.0;
    int position;
    while (running){
        //Add income, expenses, delete
        char command[20];
        printf("List of Commands:\n> add income\n> add expense\n> print\n> delete\n> quit\n");
        printf("Enter command: ");
        scanf("%19s", command);

        //condition to function
        //Add Income feature
        if (strcmp(command, "add income") == 0){
            printf("Enter income description: ");
            scanf("%99s", description);
            printf("Enter amount: ");
            while(scanf("%f", &amount) != 1 || amount <= 0){
                printf("Invalid input! Try again: ");
                while(getchar() != '\n'); // clear the corrupted input buffer line
            }
            while (getchar() != '\n'); // clear remaining newline character

            printf("Insert at position: ");
            scanf("%d", &position);
            add_income(head_node, position, amount, balance, description);
            printf("Income added.\n\n");
            printf("\nCurrent balance: $%.2f\n", *balance);
            printf("Budget Status: %s\n", budgetStatus(balance, expense));

            // Add Expense Feature
        } else if(strcmp(command, "add expense") == 0){
            printf("Enter expense description: ");
            scanf("%99s", description);
            printf("Enter amount: ");
            while(scanf("%f", &amount) != 1 || amount <= 0){
                printf("Invalid input! Try again: ");
                while(getchar() != '\n'); // clear the corrupted input buffer line
            }
            while (getchar() != '\n'); // clear remaining newline character

            printf("Insert at Position: ");
            scanf("%d", &position);
            add_expense(head_node, position, amount, balance, description);
            *expense += amount;
            printf("Expense added.\n\n");
            printf("\nCurrent balance: $%.2f\n", *balance);
            printf("Budget Status: %s\n", budgetStatus(balance, expense));

            // Quit the program
        } else if(strcmp(command, "quit") == 0) {
            if(*fptr != NULL) fclose(*fptr);
            *fptr = fopen("../logs/transaction_log.txt", "w");
            // fptr validation
            if (*fptr == NULL){
                printf("Error: Could not open file for saving!\n");
                running = 0;
                return;
            }
            // temporary pointer starting at head
            Node* current = *head_node;
            printf("Saving transactions to file...\n");
            while(current != NULL){
                if(!current->is_deleted){
                    if(current->is_expense == 0){
                        fprintf(*fptr, "INC|%.2f\n", current->description, current->data);
                    } else {
                        float save_amount = current->data < 0 ? -current->data : current->data;
                        fprintf(*fptr, "EXP|%.2f\n", current->description, save_amount);
                    }
                }
                current = current->next;
            }

            if(*fptr != NULL) fclose(*fptr);

            printf("Done. Exiting Program.\n");
            running = 0; //Not running

            //Delete Transaction Feature
        } else if(strcmp(command, "delete") == 0){
            printf("Which transaction you want to delete? (Enter position): ");
            scanf("%d", &position);
            delete(head_node, position, balance);
            printf("Transaction deleted\n");
            printf("\nCurrent balance: $%.2f\n", *balance);
            printf("Budget Status: %s\n", budgetStatus(balance, expense));

            // Print Transaction feature
        } else if (strcmp(command, "print") == 0) {
            print(*head_node);
            printf("\nCurrent balance: $%.2f\n", *balance);
            printf("Budget Status: %s\n", budgetStatus(balance, expense));
        }
    }
    return;
}
```


Program Flow

First, we ask users if they want to resume the previous session (using the same transaction logs file or want to create a new one). After that, it will take the user to the main program. The `transaction_log.txt` will be opened. If it doesn't exist, the file will be created so that we can save our transaction to the log file.

```
Welcome to your Personal Finance Tracker!
```

```
Would you like to resume your previous session? (y/n): █
```

From now on, users are able to type 5 commands: add income, add expense, print, delete, and quit.

```
Resuming from last session...
opening file successful!
Current Log
=====
=====
```

```
Current balance: $0.00
Budget Status: Within Budget
List of Commands:
> add income
> add expense
> print
> delete
> quit
Enter command: █
```

If the user wants to add income to the transaction, they just type in “**add income**”. Then, they will be able to put in the amount, description, and a position to insert. Finally, the transaction will be updated, as well as the balance.

```
Enter command: add income
Enter income description: salary
Enter amount: 19999
Insert at position: 1
Income added.
```

```
Current balance: $19999.00
Budget Status: Within Budget
```

The expense function works the same and will subtract the amount of expense from the balance.

If a user wants to see all the transactions, typing “**print**” will do.

```
Enter command: print
[ Transactions ]
1. salary                +19999.00          +++ i

Current balance: $19999.00
Budget Status: Within Budget
```

If they want to delete a transaction, they just need to type “**delete**,” and they will be asked for the position of the desired transaction.

```
Enter command: delete
Which transaction you want to delete? (Enter position): 1
Transaction deleted

Current balance: $0.00
Budget Status: Within Budget
```

Once they want to quit, they just need to type “**quit**” and all the transactions will be saved to `transaction_logs.txt`, ready to be resumed for the next session.

```
[ Transactions ]
1. lunch                 -60.00          +++ i
2. income01             +100000.00       +++ i
3. income02              +2.00          +++ i
4. salary                +19999.00       --- d
5. nose tea             -140.00          +++ i

Current balance: $99802.00
Budget Status: Within Budget
List of Commands:
> add income
> add expense
> print
> delete
> quit
Enter command: quit
Saving transactions to file...
Done. Exiting Program.
```

```
[c_price_souls_126@c-price-souls-126s-MacBook-Air logs % ls
transaction_log.txt
[c_price_souls_126@c-price-souls-126s-MacBook-Air logs % cat transaction_log.txt
EXP|lunch|60.00
INC|income01|100000.00
INC|income02|2.00
EXP|nose tea|140.00
```


Challenges

- The node data structure is the most complex problem I have encountered in this project. I knew how to create a node; however, I had no idea how to delete, insert at a specified position, or how to keep each element dynamically updated with each other's functions.
 - I tackled these problems by doing a lot of searching, looking for examples on the internet. My lifesaver is **Geek4Geek**, as they provide excellent resources for this project.
- Later on, the program became more complex. With one tiny change, it might cause an error. Because there are many functions and conditions that have to work together; therefore, it was really hard to tackle all of them in one day.
 - I solved this problem by testing my program and code little by little until the result was satisfactory.
- Makefile & file handle. This challenge occurred because I didn't know how to properly access files in another directory in the Makefile. Later on, I learned how to do so and solved this particular problem.

Reflection & Future Improvement

- I realized how bad my data structure was at the beginning. Even after I finished the project, I still struggle to recall the node data structure. After this, I will learn more deeply about data structures and try to solve coding problems more.
- I could be able to design how the app works, but I was struggling with how to implement it in coding. Therefore, I really need to give myself a lot more time and use more paper to translate the design to code.
- The most time-consuming part for me is debugging. There were a huge number of bugs I encountered along the project. It was a big part of coding in this project.
- Finally, this project proves how much I don't know in programming, so I will use my time off to master programming in computer science more, and will not end up at McDonald's for sure!

References

<https://www.geeksforgeeks.org/c/c-program-for-deleting-a-node-in-a-linked-list/>
<https://www.geeksforgeeks.org/c/c-program-for-inserting-a-node-in-a-linked-list/>
<https://www.geeksforgeeks.org/c/linked-list-in-c/>
https://www.youtube.com/watch?v=f1r_jxCyOI0